2a. Claude and gemini
2b. Claude through the website, and gemini through the website
2c. Prompt used:
“

Generate C++ code using objects, not functions, to implement the following requirements
The Sudoku problem, then, is this: Given a partially completed Sudoku puzzle, find the (or at least a) solution; if no solution exists, prove that.
• It is possible to solve a Sudoku puzzle entirely through logic. For example, in the above puzzle, consider the bottom right block, top right cell. Because of other numbers in the same row, column, or grid, that number cannot be 1, 2, 3, 5, 6, 7, 8, or 9, and so must be 4. By proceeding in this fashion, using these and more advanced deduction rules, it’s possible to gradually fill in the entire grid.
• That is not the approach we’re going to use. Instead, we’re going to implement a brute-force search, in which we exhaustively try possibilities until we find a solution. This will be slower to run but is easier to solve. (And by “slower to run,” we mean it'll finish in a few tenths of a second rather than a few milliseconds.)
• Here’s the idea: Suppose we have a Sudoku puzzle of which k of the 81 cells are unfilled. We identify an empty cell and deduce a list of candidates for what might go into that cell. If there are no such values (all digits are accounted for elsewhere in the same row, column, or grid), then we must have made a mistake in filling in the k cells and return a no-solution-found result. Otherwise, we try putting the first candidate value into the empty cell, and see if we can find a solution with the k+1 cells filled (i.e. a recursive call). If we get a no-solution-found to the first candidate, then we try the next, and so on, until we either find a solution (k == 81) or determine that no solution exists (all candidates report no solution, which again implies we took a wrong path somewhere earlier).
• This approach, by the way, is called depth-first search with backtracking. For each empty cell we make an attempt and see how far we can get with it. If we fail to find a solution, we back up until we find something else to try, and so on. If a solution exists, we are guaranteed to find it, because we exhaustively try all possibilities.
• You are given five files (puzzle1.txt through puzzle5.txt), each containing a Sudoku puzzle. Digits already filled in have been placed, and an underscore character (_) is used to indicate a blank square.
• Write a recursive object-oriented program to find the solution to the five puzzle files.
• The output for each solution, should include:
o The puzzle file name (e.g., puzzle1.txt)
o The puzzle from the file printed out.
o The solution to the puzzle printed out or if no solution is found, “No solution found” printed out.

• The solution may not be unique; if there is more than one solution to the puzzle, your program should print them all out.
• To help you debug your program, solution1.txt is a solution to puzzle1.txt.
“

2d. Claude’s code
“

```
// Sudoku brute-force solver (depth-first search with backtracking).
// Object-oriented design: all behavior lives in classes (no free functions
// other than main). Finds and prints ALL solutions for each puzzle file.
//
// Build: g++ -std=c++17 -O2 -o sudoku sudoku.cpp
// Run:   ./sudoku puzzle1.txt puzzle2.txt ... puzzle5.txt
//        (with no arguments it tries puzzle1.txt .. puzzle5.txt)

#include <array>
#include <fstream>
#include <iostream>
#include <string>
#include <vector>

// ---------------------------------------------------------------------------
// Grid: holds the 9x9 board and knows how to read, print, and validate cells.
// ---------------------------------------------------------------------------
class Grid {
public:
    static const int N = 9;
    static const int BOX = 3;

    Grid() { for (auto &row : cells_) row.fill(0); }

    // Reads a puzzle file: digits 1-9 are givens, '_' (or '0'/'.') is blank.
    // Returns false if the file cannot be opened.
    bool load(const std::string &filename) {
        std::ifstream in(filename);
        if (!in) return false;
        int r = 0, c = 0;
        char ch;
        while (in >> ch && r < N) {
            if (ch >= '1' && ch <= '9') cells_[r][c++] = ch - '0';
            else if (ch == '_' || ch == '0' || ch == '.') cells_[r][c++] = 0;
            else continue;  // ignore any other characters
```

```
        if (c == N) { c = 0; ++r; }
    }
    return r == N;
}

int get(int r, int c) const { return cells_[r][c]; }
void set(int r, int c, int v) { cells_[r][c] = v; }
bool isEmpty(int r, int c) const { return cells_[r][c] == 0; }

// True if value v does not already appear in the row, column, or 3x3 box.
bool canPlace(int r, int c, int v) const {
    for (int i = 0; i < N; ++i) {
        if (cells_[r][i] == v) return false;
        if (cells_[i][c] == v) return false;
    }
    int br = (r / BOX) * BOX, bc = (c / BOX) * BOX;
    for (int i = 0; i < BOX; ++i)
        for (int j = 0; j < BOX; ++j)
            if (cells_[br + i][bc + j] == v) return false;
    return true;
}

// Checks the givens for conflicts (a puzzle with duplicate givens has no solution).
bool givensConsistent() {
    for (int r = 0; r < N; ++r)
        for (int c = 0; c < N; ++c) {
            int v = cells_[r][c];
            if (v == 0) continue;
            cells_[r][c] = 0;
            bool ok = canPlace(r, c, v);
            cells_[r][c] = v;
            if (!ok) return false;
        }
    return true;
}

// Finds the first empty cell in row-major order. Returns false if full.
bool findEmpty(int &row, int &col) const {
    for (int r = 0; r < N; ++r)
        for (int c = 0; c < N; ++c)
            if (cells_[r][c] == 0) { row = r; col = c; return true; }
    return false;
}
```

```
    // Candidate list for a cell: every digit not excluded by row/column/box.
    std::vector<int> candidates(int r, int c) const {
        std::vector<int> result;
        for (int v = 1; v <= 9; ++v)
            if (canPlace(r, c, v)) result.push_back(v);
        return result;
    }

    void print(std::ostream &os) const {
        for (int r = 0; r < N; ++r) {
            for (int c = 0; c < N; ++c) {
                if (cells_[r][c] == 0) os << "_ ";
                else os << cells_[r][c] << ' ';
            }
            os << '\n';
        }
    }

private:
    std::array<std::array<int, N>, N> cells_;
};

// ---------------------------------------------------------------------------
// Solver: recursive depth-first search with backtracking over a Grid.
// Collects every solution found.
// ---------------------------------------------------------------------------
class Solver {
public:
    explicit Solver(const Grid &puzzle) : grid_(puzzle) {}

    // Runs the search; returns the number of solutions found.
    int solve() {
        solutions_.clear();
        if (!grid_.givensConsistent()) return 0;
        search();
        return static_cast<int>(solutions_.size());
    }

    const std::vector<Grid> &solutions() const { return solutions_; }

private:
    // Recursive step: fill one empty cell with each candidate in turn.
    void search() {
        int r, c;
```

```
        if (!grid_.findEmpty(r, c)) {       // k == 81: complete solution
            solutions_.push_back(grid_);
            return;
        }
        std::vector<int> cands = grid_.candidates(r, c);
        for (int v : cands) {               // no candidates -> loop skipped -> backtrack
            grid_.set(r, c, v);
            search();                       // recurse with k+1 cells filled
            grid_.set(r, c, 0);             // undo, try next candidate
        }
    }

    Grid grid_;
    std::vector<Grid> solutions_;
};

// ----------------------------------------------------------------------------
// PuzzleRunner: loads a file, prints the puzzle, solves, and reports results.
// ----------------------------------------------------------------------------
class PuzzleRunner {
public:
    explicit PuzzleRunner(const std::string &filename) : filename_(filename) {}

    void run() {
        std::cout << "==============================\n";
        std::cout << filename_ << "\n";
        Grid puzzle;
        if (!puzzle.load(filename_)) {
            std::cout << "Could not read puzzle from file.\n\n";
            return;
        }
        std::cout << "Puzzle:\n";
        puzzle.print(std::cout);

        Solver solver(puzzle);
        int count = solver.solve();

        if (count == 0) {
            std::cout << "\nNo solution found\n\n";
            return;
        }
        std::cout << "\n" << count << (count == 1 ? " solution" : " solutions")
                  << " found:\n";
        int n = 1;
```

```
        for (const Grid &g : solver.solutions()) {
            if (count > 1) std::cout << "Solution " << n++ << ":\n";
            else std::cout << "Solution:\n";
            g.print(std::cout);
            std::cout << '\n';
        }
    }

private:
    std::string filename_;
};

int main(int argc, char *argv[]) {
    std::vector<std::string> files;
    if (argc > 1) {
        for (int i = 1; i < argc; ++i) files.push_back(argv[i]);
    } else {
        for (int i = 1; i <= 5; ++i)
            files.push_back("puzzle" + std::to_string(i) + ".txt");
    }
    for (const std::string &f : files) {
        PuzzleRunner runner(f);
        runner.run();
    }
    return 0;
}
“
```

2d. Gemini’s code
“

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

class SudokuSolver {
private:
    static const int SIZE = 9;
    int grid[SIZE][SIZE];
    std::string filename;
    int solutionCount;

    // Checks if placing 'num' at grid[row][col] is valid
    bool isValid(int row, int col, int num) const {
        for (int i = 0; i < SIZE; ++i) {
            // Check row and column
            if (grid[row][i] == num || grid[i][col] == num) {
                return false;
            }
        }

        // Check 3x3 subgrid
        int startRow = (row / 3) * 3;
        int startCol = (col / 3) * 3;
        for (int r = 0; r < 3; ++r) {
            for (int c = 0; c < 3; ++c) {
                if (grid[startRow + r][startCol + c] == num) {
                    return false;
                }
            }
        }
        return true;
    }

    // Finds the next empty cell (represented by 0)
    bool findEmptyCell(int& row, int& col) const {
        for (row = 0; row < SIZE; ++row) {
            for (col = 0; col < SIZE; ++col) {
                if (grid[row][col] == 0) {
                    return true;
                }
```

```
            }
        }
        return false;
    }

    // Recursive depth-first search with backtracking
    void solveRecursive() {
        int row, col;

        // Base case: If no empty cell is left, a solution is found
        if (!findEmptyCell(row, col)) {
            solutionCount++;
            std::cout << "\nSolution #" << solutionCount << ":\n";
            printGrid();
            return;
        }

        // Try candidate numbers 1 through 9
        for (int num = 1; num <= SIZE; ++num) {
            if (isValid(row, col, num)) {
                grid[row][col] = num; // Place candidate

                solveRecursive();    // Recurse to solve next empty cell

                grid[row][col] = 0;   // Backtrack
            }
        }
    }

public:
    SudokuSolver(const std::string& file) : filename(file), solutionCount(0) {
        for (int r = 0; r < SIZE; ++r) {
            for (int c = 0; c < SIZE; ++c) {
                grid[r][c] = 0;
            }
        }
    }

    bool loadPuzzle() {
        std::ifstream inFile(filename);
        if (!inFile.is_open()) {
            std::cerr << "Error: Could not open file " << filename << "\n";
            return false;
        }
```

```
    char ch;
    for (int r = 0; r < SIZE; ++r) {
        for (int c = 0; c < SIZE; ++c) {
            if (!(inFile >> ch)) {
                std::cerr << "Error: Invalid file format in " << filename << "\n";
                return false;
            }
            if (ch == '_') {
                grid[r][c] = 0;
            } else if (ch >= '1' && ch <= '9') {
                grid[r][c] = ch - '0';
            } else {
                grid[r][c] = 0;
            }
        }
    }
    inFile.close();
    return true;
}

void printGrid() const {
    for (int r = 0; r < SIZE; ++r) {
        for (int c = 0; c < SIZE; ++c) {
            if (grid[r][c] == 0) {
                std::cout << "_ ";
            } else {
                std::cout << grid[r][c] << " ";
            }
        }
        std::cout << "\n";
    }
}

void solve() {
    std::cout << "========================================\n";
    std::cout << "Puzzle File: " << filename << "\n";
    std::cout << "----------------------------------------\n";
    std::cout << "Original Puzzle:\n";
    printGrid();

    solveRecursive();

    if (solutionCount == 0) {
```

```
            std::cout << "\nNo solution found\n";
        }
        std::cout << "========================================\n\n";
    }
};

int main() {
    std::vector<std::string> puzzleFiles = {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };

    for (const auto& file : puzzleFiles) {
        SudokuSolver solver(file);
        if (solver.loadPuzzle()) {
            solver.solve();
        }
    }

    return 0;
}
“
```

2e. Individual Analysis
Claude

The time complexity is O(9^k) in the worst case, where k is the number of empty cells, since the search tries up to nine digits per empty cell. In practice the row, column and box checks prune most branches, so far fewer nodes are visited. The space complexity is O(k + S·81), where S is the number of solutions. The recursion depth is at most k, but each frame allocates a std::vector of candidates, since it saves all 81 spots for each solution. The program runs correctly and rejects puzzles whose givens already conflict, printing “No solution found”. The weakest points are the per-call heap allocation, which slows the search, and memory that grows with the number of solutions. There are some comments, and the

Yours

The time complexity is the same, O(9^k), since it uses the same depth-first search that fills the first empty cell and tries 1 through 9 in order. It should run slightly faster, because it checks digits inline with isValid and never allocates during the search. The space complexity is O(k) since only the recursion is in memory. Each solution is printed the moment it is found. The

program runs correctly. The file reader has the values hard coded, instead of given at run time. There are very few comments, and no description of what each section is doing.

Difference

Both programs visit the same nodes in the same order, so they print identical solutions in identical order. Gemini uses constant extra memory and no heap allocation, which is better for a puzzle with a huge number of solutions. For example an empty puzzle on Claude's would cause a massive amount of memory to be allocated to store every possible solution. Gemini's is better because of the serious problem of memory being used to store the solutions.

Improvements
Checking if puzzle is valid before attempting, command line input instead of hardcoded files names, separate board and solver classes